

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA0612	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Shaik Azeem	Reg. No.:	192472307
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

Code of Conduct

I Shaik Azeem(192472307)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shaik Azeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding ; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex real world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision Making Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
-------------------	--------------------	--------------------

Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q49. A banking system uses Binary Search on sorted customer IDs, but frequent insertions disturb order.

- a) Explain how Binary Search works.
- b) Analyze the impact of unsorted data.
- c) Discuss the importance of maintaining sorted datasets.

a) Explain how Binary Search works

Binary Search is a searching algorithm that works only on a **sorted** dataset.

Steps:

1. Find the middle element of the sorted array.
2. If the middle element is the target, the search stops.
3. If the target is smaller, search the left half.
4. If the target is larger, search the right half.
5. Repeat until the element is found or the search space becomes empty.

Example:

Customer IDs: [101, 105, 110, 115, 120, 125, 130]

Search for **120**

- Step 1: Middle = **115**
- Step 2: $120 > 115 \rightarrow$ Search right half.
- Step 3: Middle = **125**
- Step 4: $120 < 125 \rightarrow$ Search left half.
- Step 5: Found **120**.

b) Analyze the impact of unsorted data

If the customer IDs are **not sorted**, Binary Search will not work correctly because it assumes the data is in ascending order.

Effects:

- May return incorrect results.
- May fail to find an existing customer ID.
- Search becomes unreliable.
- The data must be sorted before using Binary Search.

c) Discuss the importance of maintaining sorted datasets

Maintaining sorted data is important because:

- Binary Search works only on sorted data.
- Searching becomes much faster ($O(\log n)$).
- Data is easier to manage and organize.
- Improves the performance of banking applications with large customer records.

```
a = [101, 105, 110, 115, 120, 125, 130]
x = 120
```

```
l, h = 0, len(a) - 1
```

```
while l <= h:
    m = (l + h) // 2
    if a[m] == x:
        print("Found at index", m)
        break
    elif x < a[m]:
        h = m - 1
    else:
        l = m + 1
else:
    print("Not Found")
```

```
PS C:\Users\shaik\Downloads\program\python\CSA0612> py test1.py
Found at index 4
```

Q50. A drone system identifies boundary points of obstacles using brute-force convex hull computation.

- a) Explain how convex hull is determined.
- b) Analyze the time complexity.
- c) Discuss why this approach is not scalable.

a) Explain how the convex hull is determined

A **Convex Hull** is the smallest convex polygon that contains all given points.

Brute-Force Method:

1. Select every pair of points.
2. Draw a line between the two points.
3. Check whether all other points lie on the same side of the line.
4. If yes, that line is part of the convex hull.
5. Repeat for all pairs of points to obtain the boundary.

b) Analyze the time complexity

For n points:

- Every pair of points is checked $\rightarrow O(n^2)$ pairs.
- For each pair, all remaining points are tested $\rightarrow O(n)$.

Therefore,

Time Complexity = $O(n^3)$

c) Discuss why this approach is not scalable

The brute-force convex hull algorithm is **not scalable** because:

- It checks every possible pair of points.
- The number of calculations increases rapidly as the number of points grows.
- It consumes more execution time for large datasets.
- It is inefficient for real-time applications such as drone navigation.

```

p = [(0,0),(0,3),(3,3),(3,0),(1,1)]

n = len(p)
print("Hull Edges:")

for i in range(n):
    for j in range(i+1, n):
        s = 0
        ok = True
        for k in range(n):
            if k != i and k != j:
                c = (p[j][0]-p[i][0])*(p[k][1]-p[i][1]) - (p[j][1]-p[i][1])*(p[k][0]-p[i][0])
                if s == 0:
                    s = 1 if c > 0 else -1 if c < 0 else 0
                elif c * s < 0:
                    ok = False
                    break
        if ok:
            print(p[i], p[j])

```

PS C:\Users\shaik\Downloads\program\python\CSA0612> py test2.py

Hull Edges:

(0, 0) (0, 3)

(0, 0) (3, 0)

(0, 3) (3, 3)

(3, 3) (3, 0)